

PROYECTO ESCOLAR DE INGENIERÍA DE SOFTWARE

Informe de cierre EduPréstamo

Sistema para la gestión de recursos y préstamos escolares

La entrega implementa autenticación JWT, roles ADMIN y USER, inventario, solicitudes de préstamo, pruebas automatizadas, CI/CD y análisis de seguridad y calidad. Las 27 pruebas aprobaron y la cobertura superó el 80 por ciento.

Fecha de cierre

18 de septiembre de 2026

Versión

1.0.0

Resumen ejecutivo

EduPréstamo quedó implementado como una aplicación web funcional para registrar recursos escolares y controlar el ciclo de préstamo. El sistema permite crear cuentas USER, iniciar sesión mediante JWT, consultar disponibilidad, solicitar materiales y administrar aprobaciones, entregas y devoluciones. El administrador también dispone de indicadores, gestión de recursos y trazabilidad por solicitante.

La validación automática aprobó 27 pruebas en cuatro suites. Jest registró 94.86 % de statements, 85.71 % de branches, 98.64 % de functions y 96.02 % de lines. OWASP ZAP no encontró XSS, inyección SQL ni alertas altas. SonarQube aprobó el Quality Gate con cero bugs, vulnerabilidades, code smells, deuda técnica y duplicación.

Objetivo del proyecto

Implementar y validar, antes del cierre del proyecto escolar, un módulo web de préstamos de recursos con autenticación JWT, roles ADMIN y USER, cobertura automatizada mínima de 80 %, pipeline CI/CD, análisis OWASP ZAP y análisis SonarQube, conservando evidencias reproducibles de cada resultado.

El objetivo es específico porque define el módulo y sus controles; medible por la cobertura, pruebas y Quality Gate; alcanzable mediante una arquitectura modular; relevante para la administración de recursos; y delimitado por la fecha de cierre.

Alcance funcional

- Registro de cuentas públicas con rol USER fijo e inicio de sesión con JWT.
- Control de acceso para funciones administrativas y funciones personales.
- Inventario con categorías, condición, existencias, búsqueda y baja lógica.
- Solicitudes con estados PENDING, APPROVED, REJECTED, DELIVERED, RETURNED y CANCELLED.
- Actualización transaccional de existencias durante aprobación y devolución.
- Panel administrativo con resumen, recursos, solicitudes y datos del solicitante.
- Reportes técnicos de pruebas, seguridad, calidad y despliegue.

Arquitectura implementada

La solución utiliza una SPA en HTML, CSS y JavaScript, una API REST en Node.js 22 con Express 5 y una base SQLite. El prototipo se mantuvo como monolito modular para reducir dependencias externas y facilitar su evaluación. Las consultas SQL usan parámetros y las operaciones que afectan inventario se ejecutan dentro de transacciones.

Capa	Tecnología	Responsabilidad
Interfaz	HTML CSS JavaScript	Acceso, catálogo, solicitudes y administración
API	Node.js 22 y Express 5	Reglas de negocio, validación y respuestas JSON
Datos	SQLite	Usuarios, recursos, préstamos y trazabilidad
Seguridad	JWT bcrypt Helmet CORS	Autenticación, roles, contraseñas y encabezados
Entrega	Docker GitHub Actions	Entorno reproducible y validación automatizada

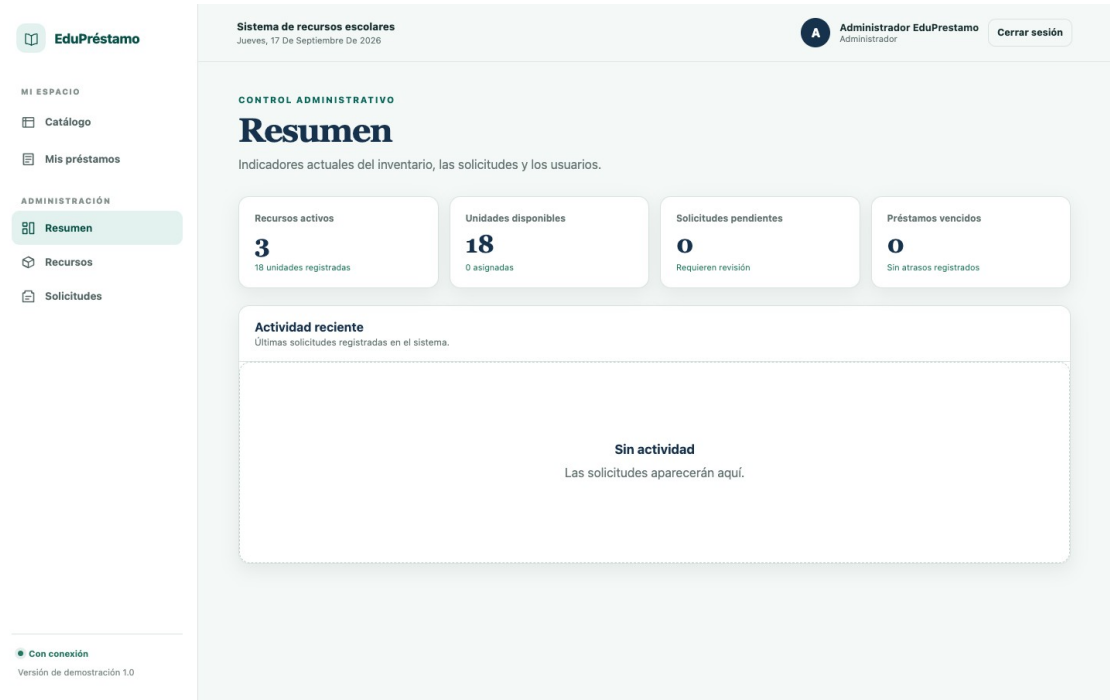


Figura 1 Panel administrativo y resumen operativo

Implementación y seguridad

Autenticación y roles

El servidor firma tokens JWT con HS256 y valida emisor, audiencia y caducidad. Las contraseñas se almacenan con bcrypt. El registro público no acepta un rol enviado por el cliente y asigna siempre USER. Las rutas administrativas exigen el rol ADMIN.

Protecciones aplicadas

- Bloqueo temporal de la cuenta después de tres intentos fallidos.
- Respuesta uniforme de autenticación para evitar enumeración de correos.
- Consultas SQL parametrizadas y transacciones para cambios de existencias.
- Helmet con política de seguridad de contenido y encabezados adicionales.
- CORS del mismo origen por defecto y cuerpo JSON limitado a 32 KiB.
- Validación de longitud, formato, fechas, cantidades, estados y roles.
- Secreto JWT obligatorio y de al menos 32 bytes en producción.

Separación de información

Cada usuario consulta únicamente sus préstamos. El administrador conserva dos vistas: Mis préstamos muestra sus solicitudes personales y Solicitudes muestra todos los registros con nombre y correo del solicitante. Esta separación evita confundir la actividad administrativa con la actividad personal.

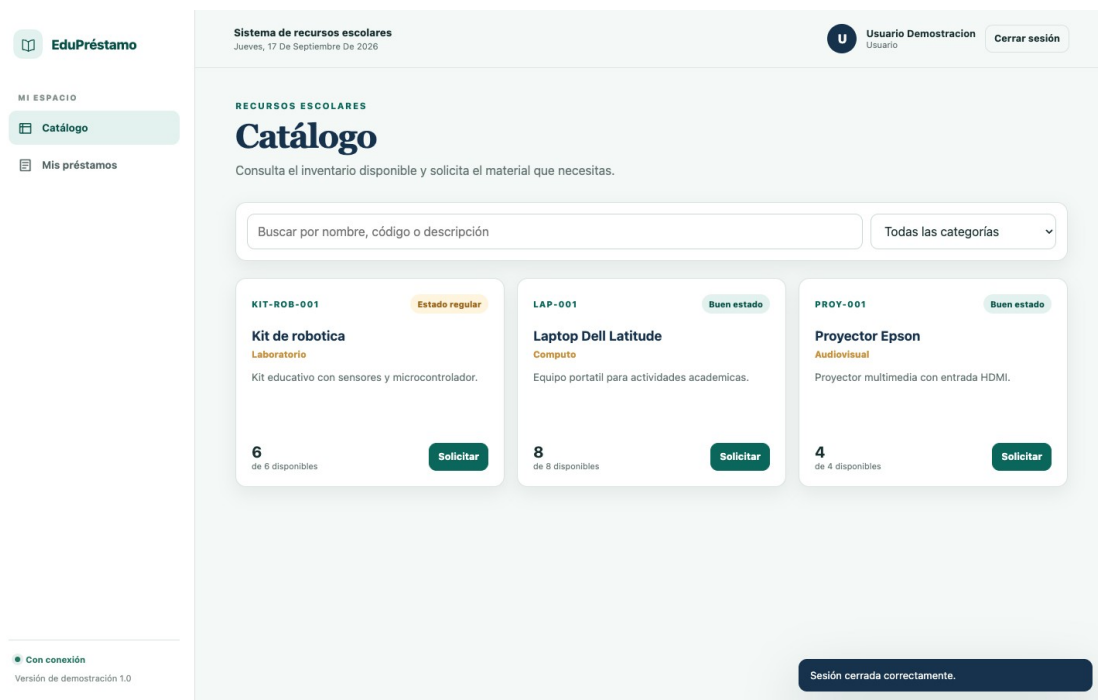


Figura 2 Catálogo disponible para una cuenta USER

Pruebas automatizadas

Las pruebas se ejecutaron con Jest y Supertest en una instalación limpia. Cubren autenticación, autorización, privacidad entre cuentas, recursos, préstamos, estados, sobreasignación, reportes y manejo de errores. El umbral global configurado es 80 %.

Métrica	Resultado	Umbral	Estado
Statements	94.86 %	80 %	Aprobado
Branches	85.71 %	80 %	Aprobado
Functions	98.64 %	80 %	Aprobado
Lines	96.02 %	80 %	Aprobado

Resultado de ejecución: 4 suites aprobadas, 27 pruebas aprobadas y 0 fallidas. La cobertura publicada corresponde al backend y no incluye src/public, src/server.js ni src/seed.js.

Integración y entrega continuas

El workflow de GitHub Actions se ejecuta en pushes, pull requests y de forma manual. La primera etapa instala dependencias, ejecuta ESLint y Jest, y publica la cobertura. La segunda construye la imagen, inicia un entorno efímero, espera el healthcheck y ejecuta ZAP. La tercera envía el análisis a Sonar cuando existe un token configurado.

1. Quality ejecuta npm ci, lint y pruebas con cobertura.
2. Container security construye y valida la imagen de producción.
3. El despliegue de prueba responde en GET /health.
4. OWASP ZAP escanea el entorno autorizado y conserva sus reportes.
5. Sonar analiza fuentes, pruebas y LCOV cuando se configura SONAR_TOKEN.

Las etapas principales fueron validadas localmente. El repositorio aún no está conectado a GitHub, por lo que no se declara una ejecución remota ni una URL inexistente. Al publicar el repositorio, el siguiente push iniciará el workflow configurado.

Resultados de OWASP ZAP

Severidad	Tipos de alerta	Interpretación
Alta	0	Sin hallazgos
Media	2	CSRF sin cookies y HTTP local
Baja	0	Sin hallazgos

Severidad	Tipos de alerta	Interpretación
Informativa	4	Autenticación y comportamiento de caché

ZAP 2.17.0 no detectó alertas de XSS, inyección SQL ni severidad alta. La alerta anti CSRF se revisó considerando que la autenticación se transmite mediante JWT Bearer y no mediante una cookie de sesión automática. La alerta HTTP corresponde exclusivamente al entorno local. Un despliegue público debe usar TLS.

El spider cubrió la superficie pública y el endpoint de inicio de sesión. Las rutas protegidas también se ejercitaron mediante pruebas HTTP automatizadas. Se conserva como mejora pendiente un escaneo API autenticado para USER y ADMIN.

Resultados de SonarQube

Métrica	Primer análisis	Resultado final
Quality Gate	Aprobado	Aprobado
Bugs	0	0
Vulnerabilidades	0	0
Code smells	7	0
Deuda técnica	63 minutos	0 minutos
Duplicación	0.0 %	0.0 %
Cobertura Sonar	91.7 %	91.6 %
Hotspots	4	4 revisados

La refactorización eliminó los siete code smells y toda la deuda técnica reportada. Sonar mantuvo calificaciones A en fiabilidad, seguridad y mantenibilidad. Los cuatro hotspots corresponden a dos credenciales demo, un falso positivo en la función de validación de contraseña y una expresión regular limitada a entradas de 254 caracteres.

Comparación entre planificación y ejecución

Elemento	Planificado	Ejecutado	Causa e impacto
Base de datos	MySQL	SQLite	Facilitó una demostración reproducible sin servidor externo. La migración queda prevista.
Roles	Estudiante docente administrador	USER y ADMIN	Se consolidaron perfiles operativos para cumplir el alcance básico sin duplicar permisos.
Módulos	Inventario préstamos historial reportes	Implementados	El historial se representa mediante estados y marcas de tiempo de cada préstamo.
Cobertura	Mínimo 80 %	85.71 % a 98.64 % según métrica	El umbral se superó en todas las métricas de backend.
Seguridad	JWT roles ZAP	Implementado y analizado	Los hallazgos locales quedaron documentados con su tratamiento.
Calidad	SonarQube	Quality Gate aprobado	La primera revisión permitió eliminar siete code smells.
CI CD	Automatización y despliegue de prueba	Workflow y entorno efímero implementados	La ejecución remota requiere conectar el repositorio a GitHub.
Registro	Módulo básico de personas	Autoservicio USER	El servidor impide que el cliente se asigne privilegios ADMIN.

Lecciones aprendidas

La evidencia debe generarse durante el desarrollo

Configurar pruebas, cobertura y reportes desde el inicio permite detectar regresiones y evita reconstruir resultados al cierre. Los archivos originales son más confiables que capturas aisladas porque conservan métricas, versiones y detalles técnicos.

La seguridad requiere revisión humana

ZAP y Sonar identifican señales, pero su severidad depende del contexto. La alerta CSRF cambia de tratamiento según el mecanismo de sesión, y los hotspots de credenciales demo no equivalen automáticamente a secretos de producción. Cada hallazgo necesita una decisión documentada.

Los roles deben reflejarse en la interfaz

Una API puede filtrar datos correctamente y aun así confundir al usuario si la interfaz usa la ruta equivocada. Separar Mis préstamos de Solicitudes administrativas aclaró la propiedad de cada registro y mejoró la trazabilidad.

La portabilidad influye en la arquitectura

SQLite simplificó la instalación y las pruebas del prototipo. La decisión reduce la carga operativa de la entrega, aunque un sistema institucional con concurrencia y copias de seguridad deberá migrar a un servicio de base de datos administrado.

Plan de mejora continua

Plazo	Mejora	Acción	Indicador
30 días	GitHub y protección de ramas	Publicar el repositorio, configurar secretos y exigir el workflow en cada pull request.	100 % de cambios con pipeline aprobado
30 días	Sesiones	Migrar el JWT de localStorage a cookie HttpOnly Secure SameSite y agregar rate limiting.	0 tokens accesibles desde JavaScript y respuesta 429 probada
60 días	Seguridad API	Publicar OpenAPI y ejecutar ZAP autenticado para USER y ADMIN.	0 alertas altas y 0 medias sin revisión
90 días	Persistencia	Migrar a MySQL o PostgreSQL administrado y automatizar respaldos.	RPO máximo de 24 horas y restauración probada
90 días	Interfaz	Agregar pruebas de navegador para registro, roles y flujo completo de préstamo.	Flujos críticos aprobados en cada release
6 meses	Predicción de demanda	Evaluar un modelo que anticipe recursos con mayor demanda usando historial suficiente.	MAPE menor o igual a 20 % frente a una línea base

Limitaciones y riesgos restantes

- El pipeline remoto todavía no tiene una ejecución porque falta conectar el repositorio a GitHub.
- El escaneo ZAP público no reemplaza un escaneo autenticado de todos los endpoints protegidos.
- El JWT se conserva en localStorage y debe migrarse antes de un despliegue público.

- Las credenciales demo deben eliminarse fuera de la presentación académica.
- SQLite es adecuado para el prototipo, pero requiere una estrategia adicional de concurrencia y respaldo en producción.

Cierre

EduPréstamo cumple el alcance funcional y técnico solicitado para la entrega académica. La aplicación implementa autenticación JWT, roles, registro USER, inventario y préstamo de recursos. Las pruebas superan el umbral de cobertura y las herramientas de seguridad y calidad producen evidencia verificable. Las limitaciones pendientes están convertidas en acciones medibles dentro del plan de mejora.

Ubicación de las evidencias

Evidencia	Ruta dentro del repositorio
Cobertura Jest	reports/coverage/
Resumen de pruebas	reports/unit/test-summary.md
OWASP ZAP	reports/security/zap/
SonarQube	reports/quality/sonarqube/
CI CD	reports/cicd/local-validation.md
Despliegue	reports/deployment/container-evidence.md
Workflow	.github/workflows/ci-cd.yml